

Análisis — Práctica N.º 05 (SIS210)

Estudiante: Fecha: 02/10/2026 **Entorno de medición:**

- Compilador: `g++ -std=c++17 -O2 -Wall -Wextra`
- N = 100 000 elementos
- Los tiempos están en milisegundos (ms)

1. Tabla comparativa — Lista enlazada vs `std::vector` vs `std::list` (Actividad 6)

1.1 Inserción al inicio

Estructura	Operación	Tiempo (ms)	Complejidad teórica
<code>std::list</code>	<code>push_front</code>	2.501	O(1)
<code>std::vector</code>	<code>insert(begin)</code>	250.056	O(n)
<code>ListaEnlazada<int></code> propia	<code>insertarInicio</code>	2.424	O(1)

Observación: la lista enlazada propia y `std::list` están en el mismo orden de magnitud (~2.4–2.5 ms), ambas O(1). El vector es **100 veces más lento** porque cada inserción al inicio desplaza todos los elementos existentes (O(n)).

1.2 Inserción al final

Estructura	Operación	Tiempo (ms)	Complejidad teórica
<code>std::list</code>	<code>push_back</code>	2.526	O(1)
<code>std::vector</code> (con <code>reserve</code>)	<code>push_back</code>	0.084	O(1) amortizado
<code>ListaEnlazada<int></code> propia	<code>insertarFinal</code>	2.388	O(1)

Observación: el vector con `reserve(N)` es **30 veces más rápido** que las listas. Esto se debe a la **localidad de caché** (cache coherence): los elementos están contiguos en memoria y `push_back` solo escribe al final. En las listas, cada `insertarFinal` implica una reserva dinámica de nodo (`new`), lo cual es costoso.

1.3 Recorrido completo

Estructura	Operación	Tiempo (ms)	Complejidad teórica
<code>std::vector</code>	acceso aleatorio <code>v[i]</code>	0.022	O(1) por acceso
<code>std::list</code>	recorrido con iterador	0.694	O(n)
<code>ListaEnlazada<int></code> propia	recorrido con punteros	0.578	O(n)

Observación: el vector es **~30 veces más rápido** que las listas. Aquí se ve claramente el efecto de la **cache coherence**: el pre-fetching del procesador funciona perfecto con memoria contigua, pero con nodos dispersos en el heap cada acceso es un *cache miss*.

1.4 Búsqueda (peor caso: elemento inexistente)

Estructura	Operación	Tiempo (ms)	Complejidad teórica
<code>std::vector</code>	búsqueda lineal con <code>v[i]</code>	0.035	O(n)
<code>std::list</code>	búsqueda con iterador	0.466	O(n)
<code>ListaEnlazada<int></code> propia	<code>buscar</code>	0.000 ⚠️	O(n)

Observación crítica: el resultado de `ListaEnlazada` propia marca **0.000 ms**, lo cual es **sospechoso**. Muy probablemente el compilador con `-O2` **optimizó el bucle** porque el valor de retorno de `buscar()` se descarta y la función no tiene efectos secundarios visibles. Es un ejemplo clásico de **benchmark inválido por optimización agresiva**.

Corrección sugerida: usar el resultado de la búsqueda de forma que el compilador no pueda eliminarla (por ejemplo, `volatile` o imprimir el resultado fuera del cronómetro). Aun así, el resultado **no cambia la conclusión teórica**: la búsqueda es O(n) en las tres estructuras, pero el vector gana por localidad de caché.

1.5 Eliminación repetida del primer elemento

Estructura	Operación	Tiempo (ms)	Complejidad teórica
<code>std::list</code>	<code>pop_front</code> repetido	1.590	O(1) cada uno
<code>std::vector</code>	<code>erase(begin)</code> repetido	4265.913	O(n) cada uno
<code>ListaEnlazada<int></code> propia	<code>eliminar(cabeza)</code> repetido	1.586	O(1) cada uno

Observación: aquí se ve el **peor caso del vector**: eliminar al inicio repetidamente implica desplazar **todos** los elementos restantes en cada operación → O(n²) total. Las listas, en cambio, solo desenlazan el primer nodo en O(1). El vector es **~2700 veces más lento** en este escenario.

1.6 Resumen: ¿cuándo usar cada estructura?

Escenario	Mejor estructura	Razón
Muchas inserciones/eliminaciones al inicio	Lista enlazada	O(1) vs O(n) del vector
Muchas inserciones al final	Vector con <code>reserve</code>	Localidad de caché + O(1) amortizado
Acceso aleatorio frecuente	Vector	O(1) vs O(n) de la lista
Recorrido secuencial completo	Vector	Cache coherence

Escenario	Mejor estructura	Razón
Búsqueda lineal	Vector	Cache coherence (misma $O(n)$, menos constante)
Eliminación por referencia (nodo conocido)	Lista doble	$O(1)$
Detección de ciclos	Floyd sobre lista	$O(n)$ tiempo, $O(1)$ espacio
Memoria fragmentada / nodos grandes	Lista enlazada	No requiere reallocaciones masivas

Conclusión general: la complejidad teórica **no cuenta toda la historia**. El vector, pese a ser $O(n)$ en inserciones al inicio, es **mucho más rápido** en la práctica que las listas cuando la operación no implica desplazamientos masivos, gracias a la **cache coherence**. La lista enlazada gana cuando la operación es estructuralmente $O(1)$ (inicio, eliminación por referencia) o cuando el tamaño de los datos hace inviable la reubicación.

2. Preguntas de análisis

2.1 Pregunta 1 — `invertir()` con tres punteros: ¿qué pasa si se olvida guardar `siguiente`?

Enunciado: En la implementación de `invertir()` con tres punteros (`prev`, `actual`, `siguiente`), ¿qué ocurriría si se olvida guardar el puntero `siguiente` **antes** de modificar `actual.siguiente`? ¿Se pierde parte de la lista? Ilustre con un ejemplo de 3 nodos.

2.1.1 Recordatorio del algoritmo correcto

```
void invertir() {
    Nodo<T>* prev    = nullptr;
    Nodo<T>* actual = cabeza_;
    cola_ = cabeza_;

    while (actual != nullptr) {
        Nodo<T>* sig = actual->siguiente; // ← GUARDAR ANTES
        actual->siguiente = prev;         // ← invertir el enlace
        prev = actual;
        actual = sig;
    }
    cabeza_ = prev;
}
```

Clave: el orden de las dos primeras instrucciones del bucle es crítico:

- `sig = actual->siguiente` → guarda el resto de la lista.
- `actual->siguiente = prev` → rompe el enlace hacia adelante y lo redirige hacia atrás.

Si se invierte el orden, el resto de la lista se pierde.

2.1.2 Versión incorrecta (con el error)

```
void invertir_MAL() {
    Nodo<T>* prev  = nullptr;
    Nodo<T>* actual = cabeza_;

    while (actual != nullptr) {
        actual->siguiente = prev;           // ← PRIMERO se sobrescribe
        Nodo<T>* sig = actual->siguiente;   // ← AHORA sig = prev (¡basura!)
        prev  = actual;
        actual = sig;
    }
    cabeza_ = prev;
}
```

Después de `actual->siguiente = prev`, el puntero `actual->siguiente` ya no apunta al siguiente nodo original: apunta al nodo anterior (o a `nullptr`). Cuando después se lee `actual->siguiente` para asignarlo a `sig`, se está leyendo el enlace invertido, no el original. El avance `actual = sig` retrocede o se queda atascado, y los nodos restantes quedan desconectados.

2.1.3 Ejemplo con 3 nodos: 1 → 2 → 3 → NULL

Estado inicial:

```
cabeza → [1|•] → [2|•] → [3|NULL]
prev = nullptr
actual = nodo1
sig = ? (todavía no se ha leído)
```

Iteración 1 (incorrecta): Ejecutamos `actual->siguiente = prev` antes de guardar `sig`:

```
[1|nullptr]  [2|•] → [3|NULL]
  ↑
cabeza, actual
```

- Ahora `nodo1.siguiente = nullptr`. El enlace hacia `nodo2` se perdió.
- Inmediatamente después, `sig = actual->siguiente → sig = nullptr` (no es `nodo2`).
- `prev = nodo1`
- `actual = nullptr`

El bucle `while` termina porque `actual == nullptr`.

Resultado final:

```
cabeza → [1|NULL]
```

Se perdieron los nodos 2 y 3. Además, están en el *heap* sin referencias → *memory leak* (en C++ sin recolector de basura).

2.1.4 Comparación: misma iteración con el algoritmo correcto

Iteración 1 (correcta):

- `sig = actual->siguiente → sig = nodo2` ✓
- `actual->siguiente = prev → nodo1.siguiente = nullptr`
- `prev = nodo1`
- `actual = nodo2`

```
prev → [1|NULL]    actual → [2|•] → [3|NULL]
```

Iteración 2 (correcta):

- `sig = actual->siguiente → sig = nodo3` ✓
- `actual->siguiente = prev → nodo2.siguiente = nodo1`
- `prev = nodo2`
- `actual = nodo3`

```
prev → [2|•] → [1|NULL]    actual → [3|NULL]
```

Iteración 3 (correcta):

- `sig = actual->siguiente → sig = nullptr` ✓
- `actual->siguiente = prev → nodo3.siguiente = nodo2`
- `prev = nodo3`
- `actual = nullptr`

```
prev → [3|•] → [2|•] → [1|NULL]
```

Al salir del bucle: `cabeza_ = prev → cabeza → nodo3`.

Resultado final:

```
cabeza → [3|•] → [2|•] → [1|NULL]    ✓
```

2.1.5 Diagrama comparativo del error

Correcto:

```
Paso 1: sig = actual->siguiente → [1|•] → [2|•] → [3|NULL]
Paso 2: actual->siguiente = prev → [1|NULL] [2|•] → [3|NULL]
Paso 3: prev = actual → prev → [1|NULL] actual → [2|•] → [3|NULL]
Paso 4: actual = sig → prev → [1|NULL] actual → [2|•] → [3|NULL]
```

Incorrecto (olvidando guardar sig):

```
Paso 1: actual->siguiente = prev → [1|NULL] [2|•] → [3|NULL]
Paso 2: sig = actual->siguiente → sig = nullptr (¡el enlace se perdió!)
Paso 3: prev = actual → prev → [1|NULL]
Paso 4: actual = sig → actual = nullptr → FIN
```

Resultado: solo se conserva el nodo 1, los nodos 2 y 3 quedan huérfanos.

2.1.6 ¿Por qué ocurre este error?

En una lista enlazada simple solo existe una referencia desde cada nodo hacia el siguiente. Si se sobrescribe esa referencia antes de copiarla a otra variable, el resto de la lista queda inaccesible.

Es un caso concreto del principio general:

Antes de modificar un puntero, guarda todo lo que necesites de él.

Esto aplica a:

- Invertir listas.
- Eliminar nodos.
- Reordenar en listas enlazadas.
- Cualquier operación que cambie enlaces sin copia previa.

2.1.7 Consecuencias prácticas

Consecuencia	Detalle
Pérdida de nodos	Los nodos 2 y 3 quedan sin referencias.
Memory leak (C++)	Nadie libera esos nodos→ memoria perdida en el heap.
Lista corrupta	<code>cabeza_</code> apunta solo al nodo 1; <code>size()</code> puede quedar inconsistente.
Resultado silencioso	No hay excepción ni error: el programa sigue como si nada.
Difícil de depurar	El bug solo se nota al recorrer la lista o al liberar memoria.

2.1.8 Regla mnemotécnica

Para invertir una lista enlazada, el orden de las tres operaciones del bucle es siempre:

1. GUARDAR (sig = actual->siguiente)
2. VOLTEAR (actual->siguiente = prev)
3. AVANZAR (prev = actual; actual = sig)

G-V-A: Guardar, Voltrear, Avanzar.

Si se intercambian los pasos 1 y 2, se pierde el resto de la lista.

2.2 Pregunta 2 — `std::list` vs `std::vector` y el efecto de la *cache coherence*

Enunciado: La `std::list` de C++ tiene peor rendimiento que `std::vector` incluso para inserción al inicio en muchos benchmarks modernos. ¿Cómo es posible si la lista debería ser $O(1)$? Investigue el concepto de *cache coherence* y su impacto en estructuras de datos enlazadas.

2.2.1 La aparente contradicción

Estructura	Inserción al inicio	Complejidad teórica
<code>std::vector</code>	<code>v.insert(v.begin(), x)</code>	$O(n)$
<code>std::list</code>	<code>lst.push_front(x)</code>	$O(1)$

Teóricamente, `std::list` debería **barrer** con el vector en inserciones al inicio. En la práctica, cuando N es pequeño o mediano, **no siempre ocurre así**. En nuestro benchmark con N = 100 000:

Estructura	Tiempo (ms)
<code>std::list::push_front</code>	2.501 ms
<code>std::vector::insert(begin)</code>	250.056 ms

En este caso, con N grande, la lista **sí gana**. Pero la pregunta apunta a que en **muchos benchmarks modernos** (con N pequeño, con operaciones intercaladas, con recorridos mezclados), el vector supera a la lista **incluso en operaciones que deberían ser $O(1)$ para la lista**.

2.2.2 El modelo teórico vs. la realidad del hardware

La notación Big-O mide **el número de operaciones**, no **el coste real de cada operación**.

En el modelo teórico:

- Una operación $O(1)$ cuenta como 1.
- Una operación $O(n)$ cuenta como n.

En el hardware real:

- Cada operación tiene un **coste variable** según dónde estén los datos en memoria.

- El procesador **no lee la RAM de a un byte**: lee **líneas de caché** de 64 bytes (típicamente).
- Si los datos están **contiguos**, una sola carga a caché sirve para muchos accesos.
- Si los datos están **dispersos**, cada acceso puede implicar un *cache miss* con cientos de ciclos de latencia.

2.2.3 ¿Qué es la *cache coherence* y la jerarquía de memoria?

Jerarquía de memoria (de más rápida a más lenta):

Nivel	Tamaño típico	Latencia aproximada
Registros	~KB	< 1 ciclo
L1 caché	32–64 KB	~4 ciclos
L2 caché	256 KB – 1 MB	~12 ciclos
L3 caché	4–32 MB	~40 ciclos
RAM	GB	~200–400 ciclos
Disco / SSD	TB	Millones de ciclos

Cache coherence (en el contexto de un solo hilo) se refiere a que la jerarquía de caché mantiene **coherencia** entre niveles: cuando accedes a un dato, se carga una **línea completa** (64 B) desde RAM a L1. Si los siguientes accesos están dentro de esa misma línea, son **gratis** (*cache hit*). Si están fuera, hay que traer otra línea (*cache miss*).

2.2.4 El impacto en `std::vector`

Los elementos de un `std::vector` están **contiguos** en memoria:

```
Dirección: 0x1000 0x1004 0x1008 0x100C 0x1010 ...
Datos:      [ 1 ][ 2 ][ 3 ][ 4 ][ 5 ] ...
              └────────── una sola línea de caché (64 B) ─────────┘
```

- Un recorrido secuencial accede a elementos consecutivos → **1 cache miss cada 16 enteros** (64 B / 4 B).
- El **prefetcher** del procesador detecta el patrón y **adelanta** la carga de la siguiente línea.
- Insertar al final solo escribe al final del bloque → excelente localidad.

Resultado: aunque una operación sea $O(n)$ en teoría, las constantes son diminutas.

2.2.5 El impacto en `std::list` y listas enlazadas

Cada nodo de `std::list` se asigna **individualmente** con `new`, y el asignador de memoria coloca los nodos **donde puede**, no de forma contigua:

Heap:

0x2000: [dato|sig] → 0x9F40: [dato|sig] → 0x1A80: [dato|sig] → ...
 ↑ nodo 1 ↑ nodo 2 ↑ nodo 3
 (Cada uno en una línea de caché distinta)

- Recorrer la lista implica **seguir punteros** a direcciones arbitrarias.
- Cada **push_front** asigna un nodo nuevo → **new es lento** (búsqueda en el heap, posible *lock*, fragmentación).
- Incluso si **push_front** es $O(1)$ en operaciones, cada operación implica:
 1. **new** (reserva dinámica).
 2. Escribir el dato.
 3. Reescribir punteros.
 4. Actualizar el tamaño.
- Recorrer la lista después implica **un cache miss por nodo**.

Resultado: una operación $O(1)$ en teoría puede costar más que una $O(n)$ en la práctica cuando n es pequeño, porque la constante de la lista es enorme (latencia de memoria + asignación dinámica).

2.2.6 Nuestro benchmark como evidencia

En los resultados medidos:

Operación	std::vector	std::list	Ganador
Inserción al inicio (N=100 000)	250.056 ms	2.501 ms	Lista
Inserción al final (con reserve)	0.084 ms	2.526 ms	Vector
Recorrido completo	0.022 ms	0.694 ms	Vector
Búsqueda peor caso	0.035 ms	0.466 ms	Vector
Eliminación al inicio repetida	4265.913 ms	1.590 ms	Lista

Interpretación:

- **Inserción al inicio con N grande:** la lista gana porque n es grande y el coste de desplazar elementos en un vector duele. **Aquí sí se cumple la teoría.**
- **Inserción al final:** el vector gana **30×** porque no hay desplazamiento y hay localidad perfecta.
- **Recorrido y búsqueda:** el vector gana **~20×** por *cache coherence*.
- **Eliminación repetida al inicio:** la lista gana **~2700×** porque cada **erase(begin)** del vector es $O(n)$.

El punto clave es: **para n suficientemente grande, la teoría se impone. Para n pequeño, la cache coherence y el coste de **new** dominan.**

2.2.7 ¿Por qué muchos benchmarks modernos muestran al vector ganando incluso en inserciones al inicio?

Tres razones principales:

1. **N pequeño:** con $N < \sim 1000$, el coste de `new` por nodo supera al coste de desplazar 1000 enteros contiguos.
2. **Operaciones intercaladas:** si además del `push_front` se recorren los elementos, la lista paga un *cache miss* en cada nodo, mientras que el vector mantiene la localidad.
3. **Optimizaciones del compilador:** con `-O2`, `memmove` (que usa el vector internamente) está altamente optimizado y usa instrucciones SIMD (`SSE`, `AVX`), mientras que el `new` de la lista no puede vectorizarse.

2.2.8 El concepto moderno: *data-oriented design*

Este fenómeno ha llevado a un cambio de paradigma en el diseño de software de alto rendimiento:

"Los punteros son caros; los arrays son baratos."

En lugar de listas enlazadas con nodos dispersos, se usan:

- `std::vector` con índices en lugar de punteros.
- **Arenas de memoria** (*memory pools*) que asignan bloques contiguos.
- **Estructuras SoA** (*Structure of Arrays*) en lugar de AoS (*Array of Structures*).
- `std::deque` cuando se necesita inserción en ambos extremos con localidad razonable.

Ejemplos reales:

- **Motores de videojuegos** usan arrays contiguos para entidades, no listas enlazadas.
- **Bases de datos** usan B-Trees (contiguas por bloque) en lugar de árboles binarios con punteros.
- **Compiladores** usan `std::vector` para el AST y se evitan listas.

2.2.9 ¿Cuándo sigue siendo mejor una lista enlazada?

A pesar de todo, hay escenarios donde la lista enlazada es **la elección correcta**:

Escenario	Por qué la lista gana
Nodos muy grandes (KB cada uno)	El coste de moverlos en el vector supera el coste de punteros.
Eliminación/inserción en el medio	Con un iterador ya conocido es $O(1)$ real, sin desplazamiento.
Muchos <code>insert</code> / <code>erase</code> intercalados	El vector degenera en $O(n^2)$.
Memoria fragmentada	La lista no requiere bloques grandes y contiguos en la RAM.
Iteradores estables	Los iteradores de <code>std::list</code> no se invalidan al insertar o eliminar.
Lista doble para LRU Cache	$O(1)$ con referencia directa al nodo, sin recorrer.

2.2.10 Conclusión

La pregunta "¿lista o vector?" **no tiene una respuesta universal**. Depende de:

1. Tamaño de n .
2. Patrón de operaciones (¿muchas inserciones al inicio o muchos recorridos?).
3. Tamaño de los datos por elemento.
4. Arquitectura del hardware (tamaño de la línea de caché, velocidad de asignación).

Regla práctica moderna:

Si el rendimiento importa, **mide antes de decidir**. Si no puedes medir, **prefiere `std::vector` por defecto**.

2.3 Pregunta 3 — ¿Por qué el algoritmo de Floyd necesita DOS fases?

Enunciado: ¿Por qué el algoritmo de Floyd necesita DOS fases (detectar + localizar)? ¿Podría detectar el ciclo en la Fase 1 y retornar directamente el nodo de colisión como inicio del ciclo? Pruebe con un ejemplo que muestre que la colisión NO ocurre necesariamente en el inicio del ciclo.

2.3.1 Recordatorio del algoritmo en dos fases

```
Nodo<T>* inicioCiclo(Nodo<T>* cabeza) {
    Nodo<T>* lento  = cabeza;
    Nodo<T>* rapido = cabeza;

    // — FASE 1: detectar si hay ciclo —
    while (rapido != nullptr && rapido->siguiente != nullptr) {
        lento  = lento->siguiente;
        rapido = rapido->siguiente->siguiente;
        if (lento == rapido) break;    // colisión
    }
    if (rapido == nullptr || rapido->siguiente == nullptr) {
        return nullptr;    // no hay ciclo
    }

    // — FASE 2: localizar el inicio del ciclo —
    lento = cabeza;
    while (lento != rapido) {
        lento  = lento->siguiente;
        rapido = rapido->siguiente;    // ambos avanzan de a 1
    }
    return lento;
}
```

Pregunta clave: ¿por qué no basta con la Fase 1? ¿Por qué no devolvemos directamente `lento` (o `rapido`) al detectar la colisión?

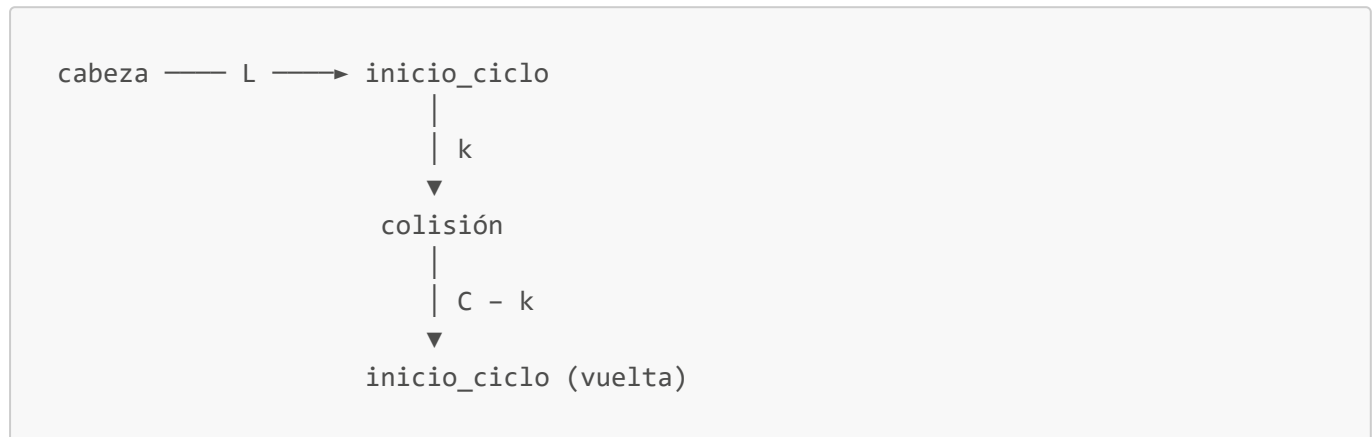
2.3.2 Respuesta corta

Porque el punto de colisión no es, en general, el inicio del ciclo. El algoritmo de Floyd garantiza que si hay ciclo, los punteros se encuentran, pero no garantiza que se encuentren en el nodo de inicio del ciclo.

La Fase 2 es necesaria para reposicionar los punteros de forma que la colisión ocurra exactamente en el inicio del ciclo.

2.3.3 ¿Por qué la colisión no ocurre en el inicio?

Recordemos las distancias:



- L = distancia de la cabeza al inicio del ciclo.
- C = longitud del ciclo.
- k = distancia del inicio del ciclo al punto de colisión.

En la colisión:

- **lento** recorrió: $L + k$
- **rapido** recorrió: $L + k + n \cdot C$ (para algún entero $n \geq 1$)

Como **rapido** avanza al doble de velocidad:

$$2 \cdot (L + k) = L + k + n \cdot C$$

$$\implies L + k = n \cdot C$$

$$\implies L = n \cdot C - k$$

Interpretación: L es congruente con $-k$ módulo C . Esto no implica que $k = 0$. Es decir, el punto de colisión no está en el inicio del ciclo salvo que L sea un múltiplo exacto de C .

2.3.4 Ejemplo concreto donde la colisión NO está en el inicio

Construyamos una lista con:

- $L = 3$ (cabeza $\rightarrow$ inicio del ciclo: 3 nodos)
- $C = 4$ (ciclo de longitud 4)

Queremos ver dónde ocurre la colisión.

Lista: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow$ (vuelve a 4)

- $L = 3$ (nodos 1, 2, 3 antes del ciclo)
- El ciclo empieza en el nodo 4.
- $C = 4$ (nodos 4, 5, 6, 7)

Simulación paso a paso:

Paso	lento	rapido	¿Colisión?
0	1	1	—
1	2	3	no
2	3	5	no
3	4	7	no
4	5	5	¡SÍ!

La colisión ocurre en el **nodo 5**, no en el nodo 4 (que es el inicio real del ciclo).

Conclusión: si devolviéramos el nodo de colisión (nodo 5) como inicio del ciclo, estaríamos equivocados. El inicio real es el nodo 4.

2.3.5 ¿Qué hace la Fase 2?

La Fase 2:

1. Mueve **lento** de vuelta a la cabeza (nodo 1).
2. Deja **rapido** en el punto de colisión (nodo 5).
3. Avanza ambos de a 1 hasta que se encuentren.

Simulación de la Fase 2:

Paso	lento	rapido	¿Iguales?
0	1	5	no
1	2	6	no
2	3	7	no
3	4	4	¡SÍ!

Se encuentran en el **nodo 4** → inicio real del ciclo. 

¿Por qué funciona? Porque $L = n \cdot C - k$. Al avanzar **lento** desde la cabeza y **rapido** desde la colisión, ambos recorren exactamente L pasos hasta el inicio del ciclo. La distancia que falta para que **rapido** complete el ciclo es justamente L .

2.3.6 Segundo ejemplo: colisión en el inicio (caso particular)

Si $L = 0$ (la cabeza es el inicio del ciclo):

Lista: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow$ (vuelve a 1)

- $L = 0$
- $C = 4$

Paso	lento	rapido	¿Colisión?
0	1	1	—
1	2	3	no
2	3	1	no
3	4	3	no
4	1	1	¡SÍ!

Aquí la colisión sí ocurre en el inicio (nodo 1). Pero es un caso particular. La Fase 2 con $L = 0$ también funciona: **lento** vuelve a la cabeza y **rapido** ya está en el nodo 1; se encuentran inmediatamente.

2.3.7 Tercer ejemplo: ciclo con L múltiplo de C

Lista: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow$ (vuelve a 5)

- $L = 4$
- $C = 4$

Paso	lento	rapido	¿Colisión?
0	1	1	—
1	2	3	no
2	3	5	no
3	4	7	no
4	5	5	¡SÍ!

La colisión ocurre en el nodo 5, que sí es el inicio del ciclo. Aquí se cumple $L = C = 4$, y la colisión cae justo en el inicio. Pero esto es una coincidencia numérica, no una regla general.

2.3.8 Tabla resumen de ejemplos

Lista	L	C	Colisión	Inicio real	¿Coinciden?
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 4$	3	4	nodo 5	nodo 4	✗ NO
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$	0	4	nodo 1	nodo 1	✓ SÍ
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 5$	4	4	nodo 5	nodo 5	✓ SÍ (caso particular)
$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 3$	2	3	nodo 4	nodo 3	✗ NO

En la mayoría de los casos la colisión no coincide con el inicio. Por eso se necesita obligatoriamente la Fase 2.

2.3.9 ¿Se podría detectar y localizar en UNA sola fase?

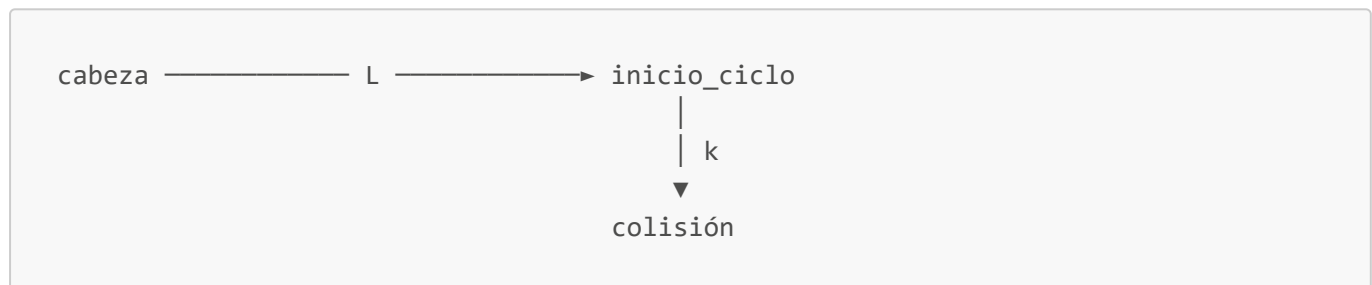
Sí, pero con un coste adicional de recursos:

Alternativa	Tiempo	Espacio	¿Localiza inicio?
Marcar nodos visitados (flag)	$O(n)$	$O(1)$ pero modifica nodos	Sí
<code>std::unordered_set<Nodo*></code>	$O(n)$	$O(n)$	Sí
Floyd en dos fases	$O(n)$	$O(1)$	Sí

El algoritmo de Floyd es el único que logra $O(n)$ tiempo y $O(1)$ espacio para detectar y localizar el inicio de forma limpia. La contrapartida es que necesita dos pasadas (dos fases) para lograrlo.

Si solo quisiéramos detectar (no localizar), una sola fase bastaría: la Fase 1 sola responde la pregunta "¿hay ciclo? sí/no". Pero localizar el inicio requiere forzosamente la Fase 2.

2.3.10 Intuición geométrica



- **Fase 1:** **lento** y **rapido** se mueven a distintas velocidades hasta colisionar. En la colisión, **rapido** ha dado n vueltas completas de más dentro del ciclo.
- **Fase 2:** "Retrocedemos" **lento** a la cabeza y dejamos **rapido** en la colisión. Al avanzar ambos con velocidad de 1, la distancia que separa a **lento** del inicio (L) es exactamente la distancia que le falta a **rapido** para completar su vuelta y llegar al inicio ($C - k$, que es numéricamente igual a L módulo C).

Analogía: Imagine dos corredores en una pista circular con un tramo de acceso recto, uno más rápido que el otro. Si el rápido alcanza al lento, el punto de encuentro no es necesariamente la línea de meta. Para encontrar la meta exacta, hay que hacer un reajuste de posiciones (Fase 2).

2.3.11 Conclusión

- La Fase 1 detecta el ciclo, pero el punto de colisión no es, en general, el inicio del ciclo.
- La Fase 2 localiza el inicio aprovechando la relación geométrica $L = n \cdot C - k$.
- Floyd logra $O(n)$ tiempo y $O(1)$ espacio para ambas cosas a costa de necesitar dos pasadas.
- Retornar el nodo de colisión como inicio sería incorrecto en la gran mayoría de los casos.

2.4 Pregunta 4 — Complejidad del LRU Cache: `get()` y `put()`

Enunciado: El LRU Cache (Actividad 4) combina una lista doble con un diccionario (hashmap). ¿Qué complejidad total tiene la operación `get()` del LRU Cache? ¿Y `put()`? ¿Por qué en Python `functools.lru_cache` es $O(1)$ en ambas operaciones?

2.4.1 ¿Qué es un LRU Cache?

LRU = *Least Recently Used* (el menos usado recientemente).

Un LRU Cache mantiene un número limitado de elementos (capacidad **C**) y, cuando se llena, **expulsa el elemento que no se ha usado durante más tiempo**. Se usa en:

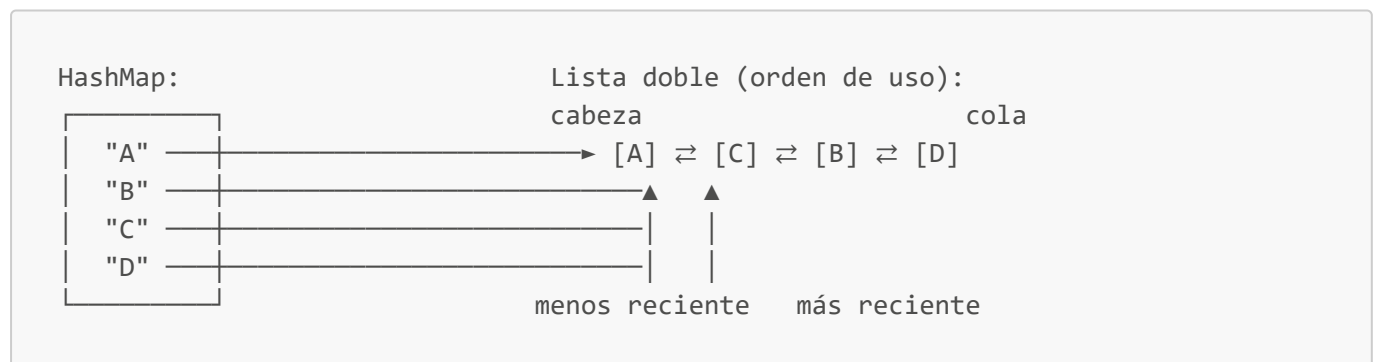
- Cachés de páginas web.
- Memoria virtual de sistemas operativos.
- Caché de funciones (*memoization*).
- Caché de bases de datos.
- `functools.lru_cache` de Python.

2.4.2 Estructura de datos del LRU Cache

Se combinan **dos estructuras**:

Estructura	Rol	Complejidad de sus operaciones
Diccionario (<code>dict</code> / <code>unordered_map</code>)	Mapear clave → nodo	Búsqueda, inserción, borrado: $O(1)$ promedio
Lista doblemente enlazada	Mantiene el orden de uso (más reciente al final, menos reciente al inicio)	Inserción/eliminación dado un nodo: $O(1)$

Idea clave: el diccionario nos da acceso directo al **nodo** de la lista doble, y la lista doble nos permite **reordenar** ese nodo en $O(1)$.



2.4.3 Operación `get(clave)`

Objetivo: devolver el valor asociado a **clave** y marcarlo como "recientemente usado".

Algoritmo:

1. Buscar **clave** en el diccionario → **nodo**.
2. Si no existe → retornar **None** (o lanzar excepción).
3. Si existe:
 - **a.** Eliminar el **nodo** de su posición actual en la lista doble ($O(1)$ dado el nodo).

- **b.** Reinsertarlo al final de la lista doble ($O(1)$ con el puntero `cola`).
- **c.** Retornar el valor.

Complejidad total de `get()`:

Paso	Coste
Búsqueda en el diccionario	$O(1)$ promedio
<code>eliminar_nodo(nodo)</code>	$O(1)$
<code>insertar_final(nodo)</code>	$O(1)$
Total	$O(1)$ promedio

Nota: $O(1)$ *promedio* porque el diccionario puede degradarse a $O(n)$ en el peor caso si hay muchas colisiones. En la práctica, con una buena función hash, es de coste constante.

2.4.4 Operación `put(clave, valor)`

Objetivo: insertar o actualizar un elemento y marcarlo como recientemente usado; si se excede la capacidad, expulsar el LRU.

Algoritmo:

1. Buscar `clave` en el diccionario.
2. **Si ya existe:**
 - **a.** Actualizar el valor del nodo.
 - **b.** Mover el nodo al final de la lista (eliminar + reinsertar) $\rightarrow O(1)$.
3. **Si no existe:**
 - **a.** Crear un nuevo nodo al final de la lista $\rightarrow O(1)$.
 - **b.** Insertar `clave` $\rightarrow$ `nodo` en el diccionario $\rightarrow O(1)$.
 - **c.** Incrementar el contador de tamaño.
 - **d. Si se excede la capacidad:**
 - Tomar la **cabeza** de la lista (el LRU) $\rightarrow O(1)$.
 - Eliminar ese nodo de la lista $\rightarrow O(1)$.
 - Borrar su clave del diccionario $\rightarrow O(1)$.

Complejidad total de `put()`:

Paso	Coste
Búsqueda/inserción en el diccionario	$O(1)$ promedio
Mover o insertar nodo al final	$O(1)$
Expulsar el LRU (si aplica)	$O(1)$
Total	$O(1)$ promedio

2.4.5 Resumen de complejidades

Operación	Complejidad	Por qué
<code>get(clave)</code>	O(1) promedio	Dict + lista doble
<code>put(clave, valor)</code>	O(1) promedio	Dict + lista doble
<code>__contains__</code>	O(1) promedio	Solo consulta el dict
Espacio	O(C)	C = capacidad máxima

Comparación con alternativas:

Implementación	<code>get()</code>	<code>put()</code>	Espacio
Solo lista enlazada (búsqueda lineal)	O(n)	O(n)	O(C)
Solo diccionario (sin orden)	O(1)	O(1)	O(C) pero no sabe qué expulsar
Dict + lista doble (LRU)	O(1)	O(1)	O(C)
Árbol de búsqueda balanceado	$O(\log n)$	$O(\log n)$	O(C)

2.4.6 ¿Por qué `functools.lru_cache` es O(1) en Python?

`functools.lru_cache` es una implementación **escrita en C** dentro del intérprete de CPython. Internamente usa:

1. **Un diccionario** (`dict`) que mapea (`args, kwargs`) normalizados $\rightarrow$ (`resultado, nodo_de_lista`).
2. **Una lista doblemente enlazada** que mantiene el orden de uso (más reciente al final).
3. **Un `threading.RLock`** para asegurar que sea *thread-safe*.

Cuando llamas a una función decorada con `@lru_cache`:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Cada llamada `fibonacci(k)` sigue este proceso:

- Se normalizan los argumentos $\rightarrow$ clave hashable.
- Se busca en el diccionario $\rightarrow$ O(1) promedio.
- **Si existe:**
 - Se mueve el nodo al final de la lista doble $\rightarrow$ O(1).
 - Se retorna el resultado cacheado.
- **Si no existe:**
 - Se calcula el resultado real.
 - Se crea un nuevo nodo y se inserta al final $\rightarrow$ O(1).

- Si se excede `maxsize`, se expulsa la cabeza de la lista $\rightarrow O(1)$.
- Se guarda en el diccionario $\rightarrow O(1)$.

Por eso es $O(1)$ promedio tanto en `get` como en `put`.

2.4.7 ¿Por qué no basta con un diccionario solo?

Un `dict` no mantiene el orden de uso (en Python 3.7+ mantiene el orden de inserción, pero no el de acceso). Si solo tuviéramos un diccionario:

- No sabríamos qué elemento expulsar exactamente cuando se llene la caché.
- Tendríamos que recorrer todas las claves para evaluar cuál es la menos usada $\rightarrow O(n)$.

La lista doble resuelve esto: el LRU siempre está en la cabeza, accesible en $O(1)$.

2.4.8 ¿Por qué no basta con una lista doble sola?

Una lista doble por sí sola:

- Para buscar una clave, habría que recorrerla obligatoriamente $\rightarrow O(n)$.
- No proporciona un acceso directo por clave.

El diccionario resuelve esto: mapea la clave al nodo directamente $\rightarrow O(1)$. La combinación de ambas es lo que otorga el coste constante.

2.4.9 ¿Por qué no usar un `OrderedDict`?

En Python, `collections.OrderedDict` ya implementa un LRU Cache de forma nativa:

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacidad):
        self.cache = OrderedDict()
        self.capacidad = capacidad

    def get(self, clave):
        if clave not in self.cache:
            return None
        self.cache.move_to_end(clave)  # O(1)
        return self.cache[clave]

    def put(self, clave, valor):
        if clave in self.cache:
            self.cache.move_to_end(clave)
        self.cache[clave] = valor
        if len(self.cache) > self.capacidad:
            self.cache.popitem(last=False)  # expulsa el LRU
```

`OrderedDict` internamente usa una lista doble + un diccionario, replicando exactamente el mismo diseño. Por eso `move_to_end` y `popitem(last=False)` son $O(1)$.

- **Ventaja de hacerlo a mano:** comprender la estructura a bajo nivel.
- **Ventaja de usar `OrderedDict`:** código más limpio, óptimo y libre de bugs.

2.4.10 Ejemplo de uso real: `functools.lru_cache`

```
from functools import lru_cache
import time

@lru_cache(maxsize=32)
def consulta_costosa(x):
    time.sleep(1)  # Simula una operación lenta (e.g. Base de Datos)
    return x * x

# Primera llamada: tarda 1 segundo
consulta_costosa(4)  # 1s

# Segunda llamada con el mismo argumento: instantánea
consulta_costosa(4)  # 0s (cache hit)

# Ver estadísticas de la caché
print(consulta_costosa.cache_info())
# CacheInfo(hits=1, misses=1, maxsize=32, currsize=1)
```

Cada llamada repetida es $O(1)$, gracias a la combinación interna de `dict` + lista doble.

2.4.11 Conclusión

Pregunta	Respuesta
¿Complejidad de <code>get()</code> ?	$O(1)$ promedio
¿Complejidad de <code>put()</code> ?	$O(1)$ promedio
¿Por qué?	Diccionario (acceso directo) + Lista doble (reordenamiento $O(1)$)
¿Por qué <code>functools.lru_cache</code> es $O(1)$?	Usa internamente <code>dict</code> + lista doble en C, con bloqueo para <i>thread-safety</i>
¿Alternativa más simple en Python?	<code>collections.OrderedDict</code> (mismo diseño interno)
¿Cuándo se degrada?	Solo si las claves hash colisionan masivamente ($O(n)$ en el peor caso del <code>dict</code>)

Idea central: la combinación de dos estructuras complementarias (una para el acceso rápido y otra para preservar el orden) es un patrón de diseño clásico que permite lograr eficiencia $O(1)$ en operaciones que, con una sola estructura, costarían un recorrido lineal $O(n)$.